

EXECUTABLE CODE

```
import pandas as pd

import numpy as np

# Load Training data

df=pd.read_csv(R'C:\Users\veere\OneDrive\Desktop\projectD12\DiseasePredictionUsingMachineLearning\Notebook\Training.csv')

df.head()

X = df.iloc[:, :-1]

y = df['prognosis']


# Train, Test split

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=20)


# Load real test data

df_test =

pd.read_csv(r'C:\Users\veere\OneDrive\Desktop\projectD12\DiseasePredictionUsingMachineLearning\Notebook\Testing.csv')
```

```

X_acutal_test = df_test.iloc[:, :-1]

y_actual_test = df_test['prognosis']


# KNeighborsClassifier

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score

knn_classifier= KNeighborsClassifier(n_neighbors=5, metric='minkowski', p=2 )

knn_classifier.fit(X_train, y_train)

y_pred= knn_classifier.predict(X_test)

print(accuracy_score(y_test, y_pred))

print("Accuracy on acutal test: ", knn_classifier.score(X_acutal_test,
y_actual_test))

result_knn=knn_classifier.score(X_acutal_test, y_actual_test)

1.0
Accuracy on acutal test: 1.0

```

```

# Support vector classifier

from sklearn.metrics import accuracy_score

from sklearn.svm import SVC # "Support vector classifier"

```

```

svc_classifier = SVC(kernel='linear', random_state=0)

svc_classifier.fit(X_train, y_train)

y_pred= svc_classifier.predict(X_test)

print(accuracy_score(y_test, y_pred))

print("Accuracy on acutal test: ", svc_classifier.score(X_acutal_test,
y_actual_test))

result_svm=svc_classifier.score(X_acutal_test, y_actual_test)

1.0
Accuracy on acutal test: 1.0

```

LogisticRegression

```

from sklearn.metrics import accuracy_score

from sklearn.linear_model import LogisticRegression

lr_classifier= LogisticRegression(random_state=0)

lr_classifier.fit(X_train, y_train)

y_pred= lr_classifier.predict(X_test)

print(accuracy_score(y_test, y_pred))

print("Accuracy on acutal test: ", lr_classifier.score(X_acutal_test, y_actual_test))

result_lr=lr_classifier.score(X_acutal_test, y_actual_test)

1.0

```

```
Accuracy on acutal test:  1.0
```

```
# GaussianNB
```

```
from sklearn.metrics import accuracy_score
```

```
from sklearn.naive_bayes import GaussianNB
```

```
nb_classifier = GaussianNB()
```

```
nb_classifier.fit(X_train, y_train)
```

```
y_pred= nb_classifier.predict(X_test)
```

```
print(accuracy_score(y_test, y_pred))
```

```
print("Accuracy on acutal test: ", nb_classifier.score(X_acutal_test, y_actual_test))
```

```
result_db=(nb_classifier.score(X_acutal_test, y_actual_test))
```

```
1.0
```

```
Accuracy on acutal test:  1.0
```

```
# DecisionTreeClassifier
```

```
from sklearn.metrics import accuracy_score
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
dt_classifier= DecisionTreeClassifier(criterion='entropy', random_state=0)

dt_classifier.fit(X_train, y_train)

y_pred= dt_classifier.predict(X_test)

print(accuracy_score(y_test, y_pred))

print("Accuracy on acutal test: ", dt_classifier.score(X_acutal_test, y_actual_test))

result_DT=dt_classifier.score(X_acutal_test, y_actual_test)

result_DT

1.0
Accuracy on acutal test:  1.0
```

Random Forest Classifier

```
from sklearn.metrics import accuracy_score

from sklearn.ensemble import RandomForestClassifier

rf_classifier = RandomForestClassifier()

rf_classifier.fit(X_train, y_train)

y_pred= rf_classifier.predict(X_test)

print(accuracy_score(y_test, y_pred))

print("Accuracy on acutal test: ", rf_classifier.score(X_acutal_test, y_actual_test))

result_RF=rf_classifier.score(X_acutal_test, y_actual_test)
```

```
1.0
Accuracy on acutal test:  1.0
```

```
symptoms_dict = {}

for index, symptom in enumerate(X):

    symptoms_dict[symptom] = index

print(symptoms_dict,end="\n\n")


input_vector = np.zeros(len(symptoms_dict))

input_vector[[symptoms_dict['itching'], symptoms_dict['skin_rash']]] = 1

print(input_vector,end="\n\n")


print(rf_classifier.predict_proba([input_vector]),end="\n\n")

print(rf_classifier.predict([input_vector]),end="\n\n")

import matplotlib.pyplot as plt

%matplotlib inline
```

```
import pandas as pd
```

```
#comp_alg={'KNN':{'result_knn'},'SVM':{'result_svm'},  
'LR':{'result_lr},'GB':{'result_db},'DT':{'result_DT'}, 'RF':{'result_RF'}}  
  
#df= pd.DataFrame(comp_alg)  
  
#print(comp_alg)  
  
#df.plot(kind='bar')  
  
#plt.show()
```

```
comp_alg={'KNN':result_knn,'SVM':result_svm,  
'LR':result_lr,'GB':result_db,'DT':result_DT, 'RF':result_RF}  
  
print(comp_alg)  
  
names = list(comp_alg.keys())  
  
values = list(comp_alg.values())  
  
plt.bar(range(len(comp_alg)), values, label=names)  
  
plt.show()
```

```
{'KNN': 1.0, 'SVM': 1.0, 'LR': 1.0, 'GB': 1.0, 'DT': 1.0, 'RF': 1.0}
```